JV32 RV32IMAC System-on-Chip Datasheet

JV32 Development Team

Version 1.3, 2026-05-18

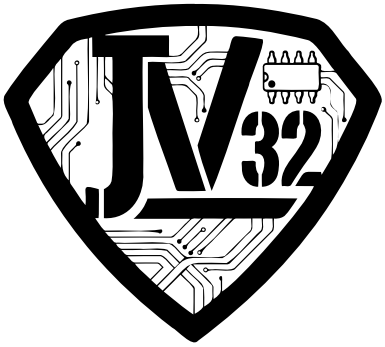


Table of Contents

This document provides comprehensive technical specifications for the JV32 System-on-Chip (SoC), a 32-bit RISC-V processor platform targeting embedded simulation and FPGA prototyping. The SoC integrates a 3-stage single-issue RV32IMAC pipeline core with 128 KB tightly-coupled instruction RAM (IRAM) and 128 KB tightly-coupled data RAM (DRAM), a merged AXI4-Lite master for out-of-TCM accesses, an AXI slave port for debug access to TCM, and three peripherals: UART (serial I/O), CLIC (interrupt controller with CLINT-compatible timer), and a simulation Magic device. The design is optimised for straightforward simulation and FPGA implementation.

Chapter 1. Introduction

1.1. Overview

The JV32 SoC is a complete minimal embedded processor system implementing the RISC-V RV32IMAC instruction set with Machine mode (M-mode) support. The system integrates:

- **RV32IMAC Processor Core** (`jv32_core`): Single-issue 3-stage pipeline (IF → EX → WB) with RVC expander, configurable fast multiply/divide/shift, branch predictor, and AMO state machine
- **Tightly Coupled Memories (TCM)**: 128 KB IRAM at `0x8000_0000` and 128 KB DRAM at `0x9000_0000`, instantiated inside `jv32_top` as byte-bank SRAM primitives with 1-cycle read latency
- **Merged AXI4-Lite Master**: Out-of-TCM accesses from both I-fetch and D-bus share one master port; D-bus has priority over I-fetch
- **AXI4-Lite Slave**: External agents (debug master, testbench DPI) can read/write TCM; core has absolute priority
- **Peripheral Subsystem** (4-slave AXI crossbar): UART, CLIC, Magic device, external catch-all
- **Interrupt System**: CLIC with CLINT-compatible 64-bit timer (`mtime`/`mtimecmp`), software interrupt (`msip`), and 16 independently-prioritised external interrupt lines

1.2. Key Features

Feature	Specification
ISA	RV32IMAC_Zicsr_Zifencei (`misa = 0x4000_1105`); optional Zba/Zbb/Zbs (`RV32B_EN=1`)
Pipeline	Single-issue 3-stage in-order (Fetch, Execute, Writeback); WB→EX same-cycle forwarding; 1-cycle load-use stall; BTFNT static branch predictor (`BP_EN=1`): 0-cycle penalty on correct prediction, 1-cycle penalty on misprediction
Instruction Memory	128 KB IRAM (TCM, 1-cycle access); out-of-TCM I-fetch via shared AXI4-Lite master
Data Memory	128 KB DRAM (TCM, 1-cycle access); IRAM is also D-accessible (IRAM address range); out-of-TCM via shared AXI4-Lite master
Privilege Modes	Machine (M-mode) only
Bus Protocol	AXI4-Lite (32-bit address, 32-bit data)

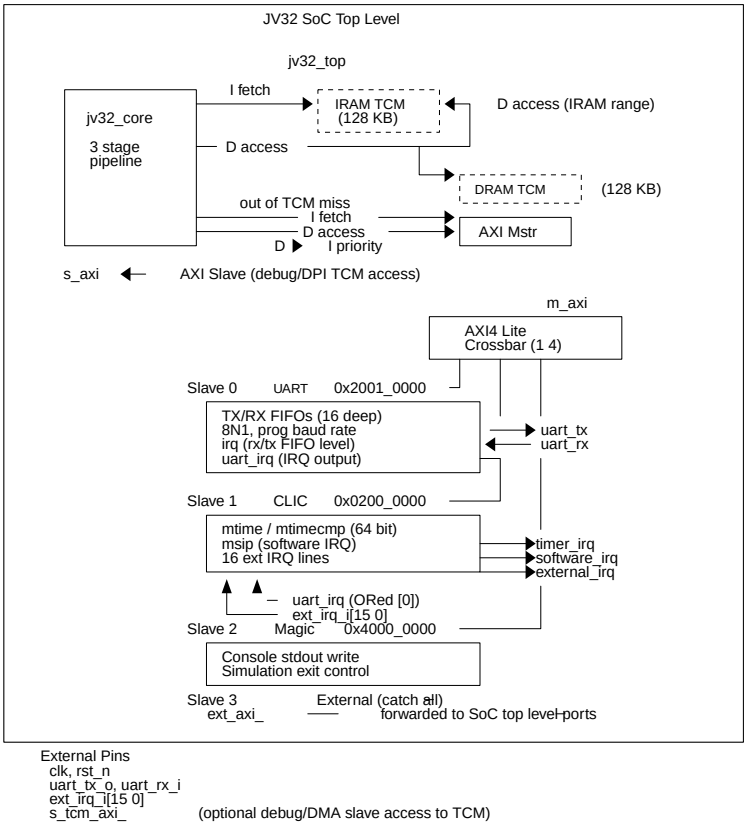
Feature	Specification
Clock Frequency	100 MHz (default, parameterisable)
Boot Address	`0x8000_0000` (IRAM base, parameterisable)
TCM Sizes	128 KB IRAM, 128 KB DRAM (each independently parameterisable, power-of-2)
Peripherals	UART (8N1, 16-deep FIFOs), CLIC/CLINT (64-bit timer, 16 external IRQs), Magic (simulation I/O)
Interrupts	Machine timer, machine software, 16 external CLIC lines (`ext_irq_i[15:0]`)
Multiply Latency	1 cycle (`FAST_MUL=1`), 1–32 cycles (`FAST_MUL=0`, CLZ-reduced on multiplier)
Divide Latency	1 cycle (`FAST_DIV=1`), 1–32 cycles (`FAST_DIV=0`, CLZ-reduced on dividend)
Implementation	RTL simulation (Verilator), FPGA synthesis supported

1.3. Document Organisation

- **Section 2**: System architecture and block diagram
- **Section 3**: Memory map and address decode
- **Section 4**: Processor core specification
- **Section 5**: TCM specification
- **Section 6**: Peripheral specifications (UART, CLIC, Magic)
- **Section 7**: AXI interconnect
- **Section 8**: Electrical and timing characteristics
- **Section 9**: Programming guide
- **Appendices**: Register quick reference, interrupt vectors, pin descriptions

Chapter 2. System Architecture

2.1. Block Diagram



2.2. System Components

2.2.1. Processor Core (`jv32_core`)

The RV32IMAC processor core implements a single-issue 3-stage in-order pipeline:

- **Instruction Fetch (IF)**: PC generation, RVC expansion, instruction fetch request
- **Execute (EX)**: Decode, register read, ALU, address calculation, CSR operations, hazard detection
- **Writeback (WB)**: Memory result, register writeback, branch redirect, exception handling

The core supports:

- Single-issue in-order execution; Slot-A all instructions; no dual-issue
- WB→EX same-cycle forwarding for RAW hazard resolution

- 1-cycle load-use stall (EX→WB stall + IF hold)
- 1-cycle branch/jump flush (EX squashes IF on misprediction or unconditional redirect)
- Multi-cycle stall for slow multiply/divide/barrel-shift when `FAST_MUL=0`/`FAST_DIV=0`/`FAST_SHIFT=0`
- AMO (LR/SC and AMO operations) via a 3-state machine (IDLE → LOAD_WAIT → STORE_WAIT)
- Configurable branch predictor (`BP_EN`, static not-taken fallback when disabled)
- CLIC-aware CSR unit with CLIC interrupt-level threshold (`mintthresh`), current level (`mintstatus`), vectored-trap base (`mtvt`), next-interrupt fast-claim (`mnxti`), and hardware tail-chaining on `mret`

2.2.2. TCM: IRAM and DRAM (`jv32_top`)

Both TCM regions are instantiated inside `jv32_top` as a single 32-bit wide `sram_1rw` primitive per region, with byte-enable write support:

- **IRAM**: instance `u_iram`, base `IRAM_BASE` (default `0x8000_0000`), size `IRAM_SIZE` (default 128 KB); 32-bit data word, 4-bit write-byte-enable
- **DRAM**: instance `u_dram`, base `DRAM_BASE` (default `0x9000_0000`), size `DRAM_SIZE` (default 128 KB); same arrangement
- **1-cycle read latency**: Address presented at cycle N returns data at cycle N+1
- **Contention resolution**: When I-fetch and D-access both target IRAM, D-access wins; I-fetch stalls 1 cycle
- **SRAM arbitration priority**: Core-D > Core-I > AXI Slave

2.2.3. Merged AXI4-Lite Master

Out-of-TCM accesses from I-fetch and D-bus share one AXI4-Lite master (`m_axi_*`):

- D-bus has priority over I-bus when both miss TCM simultaneously
- Both are served sequentially (not simultaneously)
- I-fetch AXI transaction is cancelled/replaced if a branch redirect arrives during the outstanding fetch

2.2.4. AXI4-Lite Slave

External agents (debug master, pre-simulation ELF loader via DPI) access TCM through the slave

port (`s_axi_*`):

- Core has absolute priority; slave stalls until the target SRAM is idle for the current cycle
- The slave read-data register latches SRAM output on the first granted cycle and holds it until the bus handshake completes
- For pre-simulation ELF loading, the recommended method is DPI `mem_write_byte` (direct SRAM array access), which is faster than AXI slave transactions

2.2.5. Peripheral AXI Crossbar

A 1-to-4 AXI4-Lite crossbar routes out-of-TCM transactions from `jv32_top's merged master to three named peripheral slaves (UART, CLIC, Magic) plus a fourth external catch-all slave. Address decode uses base/mask comparison (see Memory Map).

Chapter 3. Memory Map

3.1. Address Space Overview

The JV32 SoC implements a 32-bit address space (4 GB) with the following allocation:

Table 1. Complete Memory Map

Address Range	Device	Size	Description
`0x0200_0000` – `0x021F_FFFF`	CLIC	2 MB	RISC-V CLIC / CLINT-compatible interrupt controller; only first 64 KB used
`0x2001_0000` – `0x2001_00FF`	UART	256 B	Serial communication peripheral (8N1, 16-deep FIFOs); 7 registers used (0x00–0x18); 0x1C–0xFF return SLVERR
`0x4000_0000` – `0x4FFF_FFFF`	Magic Device	256 MB (aperture)	Simulation console and exit control; only 8 bytes used (CONSOLE at 0x0000, EXIT at 0x0004)
`0x8000_0000` – `0x8001_FFFF`	IRAM (TCM)	128 KB	Tightly-coupled instruction + data RAM (inside `jv32_top`)
`0x9000_0000` – `0x9001_FFFF`	DRAM (TCM)	128 KB	Tightly-coupled data RAM (inside `jv32_top`)

TCM regions (IRAM, DRAM) are not accessible via the peripheral crossbar. They are decoded inside `jv32_top` before the AXI master port. Only accesses that miss TCM are forwarded to the crossbar.

3.2. Address Decode Rules

3.2.1. TCM Decode (inside `jv32_top`)

TCM is decoded combinatorially using power-of-2 masking:

Table 2. TCM Address Decode Functions

Region	Condition	Default
IRAM	`(addr & ~(IRAM_SIZE-1)) == IRAM_BASE`	`IRAM_BASE=0x8000_0000`, `IRAM_SIZE=131072`
DRAM	`(addr & ~(DRAM_SIZE-1)) == DRAM_BASE`	`DRAM_BASE=0x9000_0000`, `DRAM_SIZE=131072`

Accesses that do not match either TCM region are forwarded to the merged AXI master and routed to the peripheral crossbar.

3.2.2. Peripheral Crossbar Decode

The AXI crossbar (`axi_xbar`) decodes using base/mask:

Table 3. Peripheral Crossbar Address Decode

Slave	Device	Base	Mask	Notes
0	UART	`0x2001_0000`	`0xFFFF_FF00`	256-byte aperture; 7 registers (DATA, STATUS, IE, IS, LEVEL, CTRL, CAPABILITY at 0x00–0x18)
1	CLIC	`0x0200_0000`	`0xFFE0_0000`	2 MB aperture; CLINT registers + CLIC interrupt array
2	Magic	`0x4000_0000`	`0xF000_0000`	256 MB aperture; only 0x1200 bytes used
3	External catch-all	`0x0000_0000`	`0x0000_0000`	Matches all addresses not decoded by slaves 0–2; forwarded to `ext_axi_*` SoC ports; returns DECERR if those ports are unconnected

Unmapped addresses return AXI `DECERR` on reads and `DECERR` on writes.

Chapter 4. Processor Core Specification

4.1. RV32IMAC ISA Support

Table 4. Supported ISA Extensions

Extension	Description
RV32I	Base integer instruction set (32-bit)
M	Integer multiply and divide, including MULH, MULHU, MULHSU, DIV, DIVU, REM, REMU
A	Atomic instructions: LR.W/SC.W, AMO{SWAP,ADD,XOR,AND,OR,MIN,MAX,MINU,MAXU}.W
C (Zca)	16-bit compressed instructions; expanded to 32-bit by `jv32_rvc` before the Execute stage
Zicsr	CSR access instructions: CSRRW, CSRRS, CSRRC, CSRRWI, CSRRSI, CSRRCI
Zifencei	Instruction-fetch fence (`FENCE.I`): flushes the IF stage and re-fetches from PC+4
Zba (optional, `RV32B_EN=1`)	Address generation: `SH1ADD`, `SH2ADD`, `SH3ADD` — shift-and-add for scaled pointer arithmetic
Zbb (optional, `RV32B_EN=1`)	Basic bit manipulation: `CLZ`, `CTZ`, `CPOP`, `ANDN`, `ORN`, `XNOR`, `MIN`/`MAX`/`MINU`/`MAXU`, `SEXT.B`, `SEXT.H`, `ZEXT.H`, `REV8`, `ORC.B`, `ROL`, `ROR`, `RORI`
Zbs (optional, `RV32B_EN=1`)	Single-bit instructions: `BCLR`, `BEXT`, `BINV`, `BSET` and their immediate variants (`BCLRI`, `BEXTI`, `BINVI`, `BSETI`)

Machine mode (M-mode) only. User mode and Supervisor mode are not implemented.

4.2. Pipeline Architecture

The core uses a single-issue 3-stage in-order pipeline:

Table 5. Pipeline Stages

Stage	Name	Function
IF	Instruction Fetch	Generate PC; present address to TCM or AXI master; pass fetched instruction through `jv32_rvc` RVC expander

Stage	Name	Function
EX	Execute	Decode opcode; read register file; execute ALU/branch/load-address/store-address/CSR; detect hazards; compute branch target; AMO state machine
WB	Writeback	Receive memory response; write result to register file; handle exceptions; redirect pipeline on branch/jump/exception/MRET/interrupt

4.3. Forwarding and Hazards

Table 6. Pipeline Hazard Handling

Hazard	Type	Description
RAW (register read-after-write)	Forwarding	WB result forwarded to EX operand inputs in the same cycle. No stall required for ALU→ALU dependency.
Load-use	Stall (1 cycle)	Load in EX presents address; data arrives in WB. The following EX-stage instruction must stall 1 cycle waiting for WB to complete. IF is also held.
Branch / Jump misprediction	Flush (1 cycle)	When EX detects a taken branch or unconditional jump, the IF instruction in flight is squashed (1 pipeline bubble).
Multi-cycle ALU (mul/div/shift)	Stall (multiple cycles)	When `FAST_MUL=0`, `FAST_DIV=0`, or `FAST_SHIFT=0`, the entire pipeline stalls until the operation completes. Serial multiply and divide use CLZ to reduce iteration count (1–32 cycles depending on operand magnitude); serial shift iterates 0–31 cycles depending on shift amount.
AMO (LR/SC, AMO operations)	Stall	Two-phase read-modify-write sequence: EX issues AXI read, waits for response (LOAD_WAIT), then issues AXI write, waits for B-channel (STORE_WAIT). Pipeline stalls throughout.
FENCE / FENCE.I	Stall / Flush	`FENCE.I` flushes the IF stage and resumes from the sequential PC; `FENCE` is a no-op in the RV32IMAC single-issue in-order pipeline (all previous stores are observable before any subsequent load).

4.4. Branch Prediction

When `BP_EN=1` (default), the core implements a **Backward-Taken / Forward-Not-Taken (BTFNT)** static branch predictor. When `BP_EN=0`, all branches are predicted not-taken and resolved in EX.

4.4.1. BTFNT Algorithm

The predictor operates at the output of the RVC expander (end of IF stage) before the IF→EX latch is updated:

1. **Opcode detection**: The 32-bit expanded instruction is inspected for `opcode == 7'b110_0011` (RISC-V BRANCH group: `BEQ`, `BNE`, `BLT`, `BGE`, `BLTU`, `BGEU`).
2. **Direction heuristic**: The B-format immediate sign bit (`instr[31]`) is tested. A set sign bit means a negative offset (branch target is behind the current PC) → **predict taken**. A clear sign bit means a positive offset (branch target is ahead) → **predict not-taken**.
3. **Redirect**: When a backward branch is predicted taken, `pc_if` is steered to the computed branch target *before* the instruction is latched into EX, and the RVC buffer is flushed to discard the speculatively fetched sequential instruction. The `bp_taken` field in the IF→EX pipeline register records whether the prediction was taken.
4. **EX correction**: When EX resolves the branch, one of three outcomes applies:
   - **Correct taken** (`bp_taken && branch_taken`): no redirect, 0-cycle penalty.
   - **Correct not-taken** (`!bp_taken && !branch_taken`): no redirect, 0-cycle penalty.
   - **Mispredicted taken** (`bp_taken && !branch_taken`): EX redirects `pc_if` to `pc_ex + instr_size` (2 for compressed, 4 for 32-bit), squashing the wrongly-fetched target instruction. 1-cycle penalty.
   - **Mispredicted not-taken** (`!bp_taken && branch_taken`): EX redirects `pc_if` to the branch target, squashing the sequential fetch. 1-cycle penalty.

4.4.2. Performance Impact

Backward branches dominate loop control flow, so BTFNT improves average CPI for loop-heavy code compared to a simple predict-not-taken strategy. The worst case is identical to predict-not-taken (1-cycle penalty).

4.4.3. BP_EN=0 Fallback

When `BP_EN=0`, the predictor logic is gated: `bp_redirect` is always 0, all branches are treated as not-taken in IF, and EX redirects the PC whenever a branch is actually taken (1-cycle penalty for every taken branch).

4.4.4. Return Address Stack (RAS)

When `BP_EN=1` and `RAS_EN=1` (default), the core adds a **2-entry circular Return Address Stack** to

predict function returns with zero penalty:

- **Push**: when a JAL or JALR with a link-register destination (rd = ra or rd = t0) is decoded in IF, the sequential return address is pushed onto the RAS.
- **Pop**: when a JALR with a link-register source and non-link destination is decoded (rs1 = ra/t0, rd ≠ ra/t0), the top-of-stack entry is used as the predicted target PC, avoiding the 1-cycle EX flush that would otherwise occur.
- RAS_EN=0: disables the RAS; returns fall back to the plain JALR not-taken path (1-cycle penalty on every return). Automatically disabled when RV32EC=1 (minimum configuration).

4.5. Performance Characteristics

Table 7. Performance Metrics

Metric	Typical Value
CPI (Cycles Per Instruction)	~1.0–1.1 (TCM hit, no branch; depends on instruction mix)
TCM Read Latency (I-fetch or D-load)	1 cycle (registered SRAM output)
Load-use Penalty	1 cycle stall
Branch Flush Penalty	0 cycles (correct BTFNT prediction) / 1 cycle (misprediction or BP_EN=0 taken branch)
Multiply Latency	1 cycle (FAST_MUL=1), 1–32 cycles (FAST_MUL=0, CLZ-reduced on multiplier)
Divide Latency	1 cycle (FAST_DIV=1), 1–32 cycles (FAST_DIV=0, CLZ-reduced on dividend)
Barrel Shift Latency	1 cycle (FAST_SHIFT=1), up to 31 cycles (FAST_SHIFT=0)
AXI Out-of-TCM Read Latency	3–10+ cycles (depends on peripheral/AXI round-trip)
IRAM contention (D vs I access)	1 extra cycle stall on I-fetch (D-path wins)

4.6. CSR (Control and Status Registers)

The core implements Machine-mode CSRs per RISC-V Privileged Specification v1.11, plus CLIC extensions:

Table 8. Implemented CSRs

Address	Name	Access	Description
0x300	mstatus	R/W	Machine status: MIE[3], MPIE[7], MPP[12:11]=11 (M-mode always)
0x301	misa	R/O	ISA and extensions: `0x4000_1105` (RV32IMAC: A[0], C[2], I[8], M[12])
0x304	mie	R/W	Machine interrupt-enable: MSIE[3], MTIE[7], MEIE[11]
0x305	mtvec	R/W	Machine trap-handler base address (bits[1:0] = mode: 00=Direct, 01=Vectored, 11=CLIC)
0x307	mtvt	R/W	CLIC vector table base address (bit[1:0] must be 0; 64-byte aligned)
0x340	mscratch	R/W	Machine scratch register (software use)
0x341	mepc	R/W	Machine exception program counter (PC of interrupted/excepted instruction)
0x342	mcause	R/W	Machine trap cause: bit[31]=interrupt flag, bits[30:0]=cause code
0x343	mtval	R/W	Machine trap value (faulting address for memory exceptions, 0 otherwise)
0x344	mip	R/W	Machine interrupt-pending: MSIP[3], MTIP[7], MEIP[11]
0x345	mnxti	R/W	CLIC next-interrupt: reads the vector address (`mtvt + 4*id`) of the highest-priority pending CLIC interrupt with level > `mintthresh`; returns 0 if no such interrupt is pending. A write (e.g. `csrrsi mnxti, 0`) also atomically claims the interrupt: updates `mcause`, sets `mintstatus.mil`, and re-enables `MIE`, allowing the handler to branch directly to the returned address (software-assisted tail-chain).
0x347	mintthresh	R/W	CLIC interrupt threshold (8-bit): only CLIC interrupts with level > mintthresh are taken

Address	Name	Access	Description
0xB00	mcycle	R/W	Machine cycle counter (lower 32 bits)
0xB02	minstret	R/W	Machine instructions-retired counter (lower 32 bits)
0xB80	mcycleh	R/W	Machine cycle counter (upper 32 bits)
0xB82	minstreth	R/W	Machine instructions-retired counter (upper 32 bits)
0xC00	cycle	R/O	Unprivileged alias of mcycle (lower 32 bits)
0xC01	time	R/O	Unprivileged alias (lower 32 bits; maps to CLIC mtime)
0xC02	instret	R/O	Unprivileged alias of minstret (lower 32 bits)
0xF11	mvendorid	R/O	Vendor ID (returns `0x0000_0000`)
0xF12	marchid	R/O	Architecture ID (returns `0x0000_0000`)
0xF13	mimpid	R/O	Implementation ID (returns `0x0000_0001`)
0xF14	mhartid	R/O	Hardware thread ID (returns `0x0000_0000`)
0xFB1	mintstatus	R/O	CLIC interrupt status: bits[31:24] = mil (machine interrupt level currently servicing)

4.7. Exception and Interrupt Handling

4.7.1. Exception Types

Table 9. Exception Causes (mcause[31]=0)

Code	Name	Description
0	Instruction address misaligned	PC not 4-byte aligned (2-byte alignment supported for RVC; 4-byte required for non-C instructions)
1	Instruction access fault	I-fetch AXI error response (SLVERR or DECERR from the AXI master)
2	Illegal instruction	Undefined or unsupported opcode

Code	Name	Description
3	Breakpoint	EBREAK instruction executed
4	Load address misaligned	Load address not naturally aligned to the access size
5	Load access fault	Load AXI error response
6	Store/AMO address misaligned	Store or AMO address not naturally aligned
7	Store/AMO access fault	Store or AMO AXI error response
8	Environment call from U-mode	ECALL executed in U-mode (not normally reached; M-mode only system)
11	Environment call from M-mode	ECALL executed in M-mode

4.7.2. Interrupt Types

Table 10. Interrupt Causes (mcause[31]=1)

Code	Name	Source
3	Machine software interrupt	CLIC MSIP register (`msip[0]`)
7	Machine timer interrupt	CLIC: `mtime >= mtimecmp`
11	Machine external interrupt	CLIC: any enabled pending external IRQ (`ext_irq_i[15:0]`) with level > `mintthresh`

4.7.3. Trap Handling Sequence

On exception or interrupt:

1. `mstatus.MPIE ← mstatus.MIE`; `mstatus.MIE ← 0` (disable interrupts)
2. `mstatus.MPP ← 2'b11` (M-mode)
3. `mepc ← PC of interrupted/excepted instruction`
4. `mcause ← {1'b1/1'b0, cause_code}`
5. `mtval ← faulting_address` (or 0)
6. PC redirected to `mtvec` (direct mode) or `mtvec + 4*cause` (vectored mode)

CLIC vectored interrupts redirect to `mtvt + 4*clic_id` (vector table indexed by the winning

interrupt's ID) and set `mintstatus.mil` to the claimed interrupt level.

Normal MRET restores: `mstatus.MIE ← mstatus.MPIE`; `mstatus.MPIE ← 1`; PC ← `mepc`.

4.7.4. Tail-Chaining (Hardware)

If, at the moment `mret` retires, a CLIC external interrupt is pending with level > `mintthresh`, the core performs a **tail-chain** rather than a full context restore:

- Fetch is redirected to `mtvt + 4*clic_id` (the next handler) — zero overhead cycles.
- `mstatus.MIE` remains 0 (already in a new handler).
- `mstatus.MPIE` is set to 1 (so the eventual chain-terminating `mret` re-enables MIE).
- `mepc` is unchanged (still holds the preempted task's return address).
- `mcause` is updated to `0x8000_000B` (machine external interrupt).
- `mintstatus.mil` is updated to the new interrupt's level.

Tail-chaining eliminates the push/pop overhead between consecutive ISRs of equal or lower priority and is fully transparent to software: the same trap handler is re-entered for each chained interrupt.

4.7.5. Software-Assisted Tail-Chain (`mnxti`)

Within a CLIC handler, software can voluntarily tail-chain to the next pending interrupt by reading `mnxti`:

```
uintptr_t next_pc = csr_swap(mnxti, 0);
if (next_pc) goto (void *)next_pc;   // chain to next handler
```

When `mnxti` is written (any write, e.g. `csrrsi mnxti, 0`) and a qualifying interrupt is pending:

- `mcause` and `mintstatus.mil` are atomically updated to the new interrupt.
- `mstatus.MIE` is re-enabled (nested interrupt within the handler).
- The read value is `mtvt + 4*clic_id` of the newly claimed interrupt (or 0 if none).

Software branches to the returned address to begin serving the next interrupt without an `mret`/re-entry cycle.

Chapter 5. TCM Specification

5.1. Overview

Both IRAM and DRAM are physically located inside `jv32_top` as a single 32-bit wide `sram_1rw` instance per region, with 4-bit byte-enable write support:

Table 11. TCM Configuration

Parameter	Default Value
IRAM base address	`IRAM_BASE = 32'h8000_0000`
IRAM size	`IRAM_SIZE = 131072` bytes (128 KB); power-of-2
DRAM base address	`DRAM_BASE = 32'h9000_0000`
DRAM size	`DRAM_SIZE = 131072` bytes (128 KB); power-of-2
Read latency	1 cycle (registered output: addr at cycle N → data at cycle N+1)
Write behaviour	Written at rising edge of cycle N; pipeline sees effect from cycle N onwards (NO_CHANGE read mode: rdata not updated on write cycles)
Data width	32 bits; 4-bit byte-enable write (`we[3:0]`); each byte lane independently writable
DPI hierarchy (IRAM)	`u_soc.u_jv32.u_iram.mem[word_index]` (byte access: `[byte_lane*8+:8]`, lane = 0..3)
DPI hierarchy (DRAM)	`u_soc.u_jv32.u_dram.mem[word_index]` (byte access: `[byte_lane*8+:8]`, lane = 0..3)

5.2. Access Priorities

When multiple requestors target the same SRAM in the same cycle:

Table 12. SRAM Arbitration Priority

Priority	Requestor	Notes
1 (highest)	Core D-path	D-access (load, store, AMO) always wins
2	Core I-path (I-fetch)	If D-path does not need the SRAM this cycle

Priority	Requestor	Notes
3 (lowest)	AXI Slave	Only granted when neither I-path nor D-path accesses the target SRAM

5.3. TCM Timing

Table 13. Timing Summary

Transaction	Behaviour
I-fetch (IRAM hit)	Addr presented at cycle N; instruction available at cycle N+1 (1-cycle latency)
D-read (IRAM or DRAM hit)	Addr presented at cycle N; data available at cycle N+1; pipeline inserts 1-cycle load stall
D-write (IRAM or DRAM hit)	Data written at rising edge of cycle N; write acknowledged at cycle N+1 (1 AXI cycle)
IRAM contention (D + I both need IRAM)	D-path wins; I-fetch stalls 1 extra cycle
Stale I-fetch after redirect	`imem_flush` signal (= `rvc_flush`) gates the TCM response; cycle N+1 response is suppressed when a branch/exception redirect occurred at cycle N

5.4. ELF Loading (Pre-Simulation)

For Verilator simulation, program images are loaded directly into TCM SRAM arrays via DPI-C before simulation starts. The testbench `tb_jv32_soc.cpp` calls `load_program()` from `elfloader.cpp`:

```
// Load ELF into TCM via DPI mem_write_byte
void mem_write_byte(uint32_t addr, uint8_t data) {
    if (addr >= IRAM_BASE && addr < IRAM_BASE + IRAM_SIZE) {
        uint32_t offset = addr - IRAM_BASE;
        uint32_t word   = offset >> 2;
        uint32_t bank   = offset & 3;
        // Writes: u_soc.u_jv32.u_iram.mem[word][bank*8+:8]
    } else if (addr >= DRAM_BASE && addr < DRAM_BASE + DRAM_SIZE) {
        uint32_t offset = addr - DRAM_BASE;
        uint32_t word   = offset >> 2;
        uint32_t bank   = offset & 3;
        // Writes: u_soc.u_jv32.u_dram.mem[word][bank*8+:8]
    }
}
```

Chapter 6. Peripheral Specifications

6.1. UART (Universal Asynchronous Receiver/Transmitter)

6.1.1. Overview

The UART peripheral provides serial communication with configurable baud rate and 8N1 format.

Table 14. UART Features

Feature	Specification
Base Address	`0x2001_0000`
Aperture	256 B (`0x2001_0000`–`0x2001_00FF`); only first 0x18 bytes used; accesses beyond 0x18 return SLVERR
Data Format	8 bits, no parity, 1 stop bit (8N1)
Baud Rate	Configurable via `BAUD_RATE` parameter (default `115_200`); compile-time fixed (`CLKS_PER_BIT = CLK_FREQ / BAUD_RATE`)
TX FIFO Depth	16 entries (parameterisable: `FIFO_DEPTH`, must be power-of-2)
RX FIFO Depth	16 entries (same as TX)
Interrupt	Level-triggered; asserted when `(IE & IS) != 0`

6.1.2. Register Map

Table 15. UART Registers (base `0x2001_0000`)

Offset	Name	Access	Description
0x00	DATA	R/W	**Write**: push byte (bits[7:0]) to TX FIFO (silently dropped if FIFO full) **Read**: pop byte from RX FIFO (returns 0 if empty)

Offset	Name	Access	Description
0x04	STATUS	R/O	**[0]** `tx_busy` — TX FIFO full / cannot accept data (same signal as `tx_full`) **[1]** `tx_full` — TX FIFO full flag **[2]** `rx_ready` — RX FIFO has data available (not empty) **[3]** `rx_overrun` — RX FIFO full (incoming bytes will be lost) NOTE: bits[1:0] reflect the same underlying `txf_full` signal; [0] exposes it as a TX-busy / write-inhibit flag, [1] as the canonical full flag.
0x08	IE (Interrupt Enable)	R/W	**[0]** `rx_ready_ie` — enable RX-ready interrupt **[1]** `tx_empty_ie` — enable TX empty interrupt
0x0C	IS (Interrupt Status)	R/O	**[0]** `rx_ready` — RX FIFO not empty (level-triggered) **[1]** `tx_empty` — TX FIFO empty (level-triggered)
0x10	LEVEL	R/W	**Read**: **[15:0]** `rx_count` — number of bytes in RX FIFO; **[31:16]** `tx_count` — number of bytes in TX FIFO **Write**: **[15:0]** baud-rate divisor (`CLKS_PER_BIT − 1`); upper 16 bits ignored. Latched immediately; any byte in flight will be corrupted.
0x14	CTRL	R/W	**[0]** `loopback_en` — internal TX→RX loopback (for testing)
0x18	CAPABILITY	R/O	**[7:0]** TX FIFO depth (= `FIFO_DEPTH` parameter, default 16) **[15:8]** RX FIFO depth (= `FIFO_DEPTH` parameter, default 16) **[31:16]** UART version (`0x0001`); combined: `{16'h0001, 8'(FIFO_DEPTH), 8'(FIFO_DEPTH)}`

Interrupt asserts when `(IE & IS) != 0`. There is no explicit interrupt-clear register; the interrupt de-asserts automatically when the FIFO condition clears (e.g. RX FIFO drained, TX FIFO refilled).

6.1.3. Baud Rate Configuration

The baud rate is determined at elaboration time:

```
CLKS_PER_BIT = CLK_FREQ / BAUD_RATE
```

Hardware constraint: `CLKS_PER_BIT >= 4` (absolute minimum for RX centre sampling).

Recommended: `CLKS_PER_BIT >= 8` for production use.

Table 16. Common Baud Rates at 100 MHz

Baud Rate	CLKS_PER_BIT	Notes
9600	10417	Very slow, large margin
115200	868	Standard; large margin
921600	109	Fast; good margin
1 Mbaud	100	Round number; good margin
12.5 Mbaud	8	Very fast; minimum recommended
25 Mbaud	4	Maximum (hardware minimum; lab use only)

Baud rates requiring `CLKS_PER_BIT < 4` will not be received correctly.

6.1.4. Interrupt Behaviour

- **RX interrupt** (`IS[0]`): Fires whenever RX FIFO is non-empty. Stays asserted until all bytes are consumed.
- **TX interrupt** (`IS[1]`): Fires whenever TX FIFO is empty. Stays asserted until software writes more data.

6.1.5. External Pins

Table 17. UART Pins

Signal	Direction	Description
`uart_tx_o`	Output	UART transmit data
`uart_rx_i`	Input	UART receive data (asynchronous to system clock)

6.2. CLIC (Core-Local Interrupt Controller)

6.2.1. Overview

The CLIC integrates the RISC-V CLINT (Core-Local Interruptor) timer and software interrupt registers with a 16-line external interrupt controller supporting per-interrupt enable, level, and priority.

Table 18. CLIC Features

Feature	Specification
Base Address	`0x0200_0000`
Aperture	2 MB (CLIC spec aperture; only CLINT registers + first 64 CLICINT entries used)
CLINT Compatibility	Full `mtime`/`mtimecmp`/`msip` register layout at standard RISC-V CLINT offsets
Timer Counter	64-bit free-running `mtime`, increments every clock cycle
Timer Compare	64-bit `mtimecmp`; timer IRQ fires when `mtime >= mtimecmp`
Software Interrupt	`msip[0]`: write 1 to assert, write 0 to clear
External Interrupt Lines	16 (`NUM_IRQ=16`), routed from `ext_irq_i[15:0]`, each with individual enable/level/priority via `CLICINT[n]`
Interrupt Outputs	`timer_irq_o`, `software_irq_o`, `clic_irq_o` (aggregated external), `clic_level_o[7:0]`, `clic_prio_o[7:0]`, `clic_id_o[4:0]` (winning IRQ index; used by CPU to compute `mtvt + 4*id`)

6.2.2. Register Map

Table 19. CLIC / CLINT Registers (base `0x0200_0000`)

Offset	Name	Access	Description
0x0000	MSIP	R/W	Machine Software Interrupt Pending. `[0]=msip`: write 1 to assert software interrupt; write 0 to clear.
0x4000	MTIME_LO	R/W	`mtime[31:0]` — lower 32 bits of 64-bit free-running timer

Offset	Name	Access	Description
0x4004	MTIME_HI	R/W	`mtime[63:32]` — upper 32 bits
0x4008	MTIMECMP_LO	R/W	`mtimecmp[31:0]` — lower 32 bits of timer compare
0x400C	MTIMECMP_HI	R/W	`mtimecmp[63:32]` — upper 32 bits. Timer IRQ fires when `mtime >= mtimecmp`.
0x1000 + n×4	CLICINT[n] (n=0..15)	R/W	Per-interrupt control word (32-bit): **[0]** `ip` (read-only) — interrupt pending; reflects `ext_irq_i[n]` (ORed with `uart_irq` for n=0) **[1]** `ie` — interrupt enable (1=enabled) **[7:2]** reserved — read as 0, writes ignored **[15:8]** reserved — read as 0, writes ignored (CLIC `attr` field **not implemented** in this RTL) **[23:16]** `ctl` — interrupt priority/level (8-bit; higher value = more urgent; used as both level and priority) **[31:24]** reserved — read as 0

MSIP, MTIME, and MTIMECMP offsets match the standard RISC-V CLINT specification for single-hart systems (compatible with `kv_clint_*` SDK functions).

6.2.3. Timer Operation

The 64-bit `mtime` counter increments by 1 every system clock cycle. The timer interrupt fires when `mtime >= mtimecmp` (unsigned 64-bit comparison), **unless `mtimecmp` equals its reset sentinel value `0xFFFF_FFFF_FFFF_FFFF`** (startup guard — prevents a spurious interrupt before software programs a compare value). Software clears the timer interrupt by writing a new, larger value to `mtimecmp`.

```
// Set timer 1 ms from now (100 MHz clock)
uint64_t now = ((uint64_t)CLIC_MTIME_HI << 32) | CLIC_MTIME_LO;
uint64_t cmp = now + 100000;   // 1 ms at 100 MHz
CLIC_MTIMECMP_LO = (uint32_t)(cmp & 0xFFFFFFFF);
CLIC_MTIMECMP_HI = (uint32_t)(cmp >> 32);
```

Always write MTIMECMP_HI (set to `0xFFFFFFFF`) before writing MTIMECMP_LO when programming a new compare value, to avoid a spurious timer interrupt in the window between the two writes on a 64-bit counter.

6.2.4. Software Interrupt Operation

Write `MSIP[0] = 1` to assert machine software interrupt immediately, `0` to clear.

6.2.5. External Interrupt Operation

Each `ext_irq_i[n]` input is gateable via `CLICINT[n].ie`. When any enabled and pending external interrupt has `ctl` (priority level) greater than the current `mintthresh` CSR value, `clic_irq_o` is asserted to the core as an M-mode external interrupt.

The CLIC asserts `clic_id_o[4:0]` with the index of the highest-priority pending enabled interrupt (highest `ctl` value wins). Both `clic_level_o[7:0]` and `clic_prio_o[7:0]` are driven with the same value — `clicint_ctl[winner]` — since this implementation does not differentiate level from priority. The CPU uses this to compute the handler address as `mtvt + 4*clic_id` (where `mtvt` is programmed to a 4-byte-pointer table). This happens automatically on interrupt entry and on hardware tail-chain at `mret`; software can also perform the same lookup by reading `mnxti`.

UART IRQ routing: the UART `irq` output (asserted when `(IE & IS) != 0`) is OR'd into `ext_irq_i[0]` inside `jv32_soc` before being presented to the CLIC. The effective `CLICINT[0].ip` thus reflects `ext_irq_i[0] | uart_irq`. Software must enable `CLICINT[0].ie` and configure `CLICINT[0].ctl` to receive UART interrupts. The external `ext_irq_i[0]` SoC pin is still available and is ORed with the UART interrupt.

6.3. Magic Device (Simulation Only)

6.3.1. Overview

The Magic Device provides simulation console I/O and simulation exit control.

The Magic Device is for simulation only. It must not be synthesised for FPGA/ASIC production.

Table 20. Magic Device Features

Feature	Specification
Base Address	`0x4000_0000`
Console Write	Offset `0x0000`: write byte (bits[7:0]) to host `stdout`

Feature	Specification
Exit Control	Offset `0x0004`: HTIF-encoded exit. Write `(exit_code << 1) | 1` to exit with the given code; `1` = pass (exit code 0). The RTL calls `sim_request_exit(value >> 1)`. Writing `0` is ignored.

6.3.2. Register Map

Table 21. Magic Device Registers (base `0x4000_0000`)

Offset	Name	Access	Description
0x0000	CONSOLE	W/O	Write character (low byte) to simulation `stdout`. Reads return 0.
0x0004	EXIT	W/O	HTIF-encoded exit control: `sim_request_exit(value >> 1)` is called when any non-zero value is written. Write `1` = pass (exit code 0); write `(code << 1) | 1` = fail with `code`. Writing `0` has no effect.

6.3.3. AXI Response Behaviour

`axi_magic` returns `OKAY` for `CONSOLE` and `EXIT` addresses. All other addresses within the Magic device AXI aperture return `SLVERR`.

6.3.4. Usage Example

```
#define MAGIC_CONSOLE  (*(volatile uint32_t *)0x40000000)
#define MAGIC_EXIT     (*(volatile uint32_t *)0x40000004)

void print_str(const char *s) {
    while (*s) MAGIC_CONSOLE = *s++;
}

void sim_pass(void) { MAGIC_EXIT = 1; }   // exit(0)
void sim_fail(int code) { MAGIC_EXIT = (code << 1) | 1; }
```

Chapter 7. AXI Interconnect and Bus Protocol

7.1. AXI4-Lite Overview

The JV32 SoC uses AXI4-Lite throughout (32-bit address, 32-bit data, no burst, no ID).

Table 22. AXI Protocol Summary

Feature	Specification
Protocol	AXI4-Lite (AMBA 4 subset: single-beat, no ARLEN/AWLEN/ARBURST, no IDs)
Data Width	32 bits
Address Width	32 bits
Write Strobes	4 bits (byte granularity)
Response Codes	`OKAY` (2'b00), `SLVERR` (2'b10), `DECERR` (2'b11)
Crossbar Topology	1 master (`jv32_top` merged output), 3 peripheral slaves

7.2. `jv32_top` AXI Master Port (`m_axi_*`)

The merged AXI master port carries all out-of-TCM traffic:

Table 23. AXI4-Lite Master Signals

Signal	Direction	Description
`m_axi_araddr[31:0]`	Output	Read address channel: address
`m_axi_arvalid`	Output	Read address valid
`m_axi_arready`	Input	Read address ready (slave accepts)
`m_axi_rdata[31:0]`	Input	Read data
`m_axi_rresp[1:0]`	Input	Read response (`OKAY`/`SLVERR`/`DECERR`)
`m_axi_rvalid`	Input	Read data valid
`m_axi_rready`	Output	Read data ready (master accepts)
`m_axi_awaddr[31:0]`	Output	Write address

Signal	Direction	Description
m_axi_awvalid	Output	Write address valid
m_axi_awready	Input	Write address ready
m_axi_wdata[31:0]	Output	Write data
m_axi_wstrb[3:0]	Output	Write byte strobes
m_axi_wvalid	Output	Write data valid
m_axi_wready	Input	Write data ready
m_axi_bresp[1:0]	Input	Write response
m_axi_bvalid	Input	Write response valid
m_axi_bready	Output	Write response ready

7.3. jv32_top AXI Slave Port (s_axi_*)

The TCM slave port has the same signal set as the master port (with directions flipped). All s_axi_* signals are exposed as SoC-level ports (s_tcm_* prefix in jv32_soc).

7.4. Crossbar Architecture

The axi_xbar module implements a 1-to-N address-decoder crossbar:

- **Address decode**: (addr & SLAVE_MASK[i]) == (SLAVE_BASE[i] & SLAVE_MASK[i]) for each slave in order; first match wins
- **Error response**: DECERR on both read and write for unmapped addresses
- **Peripheral protocol restriction**: All peripheral slaves are single-beat only; ARLEN/AWLEN are not forwarded; RLAST is hardcoded to 1'b1

7.5. Bus State Machines (inside jv32_top)

The merged AXI master is implemented as a state machine with the following states:

Table 24. AXI Master State Machine

State	Description
BUS_IDLE	Both I-fetch and D-access in TCM or idle. In IDLE, D-bus out-of-TCM check has priority over I-bus.
DAR	D-bus AXI read address phase: m_axi_araddr set to D-bus address; wait for arready.
DR	D-bus AXI read data phase: wait for m_axi_rvalid; return data to core D-path.
DAW	D-bus AXI write address+data phase: issue both AW and W channels (simultaneous when possible).
DB	D-bus AXI write response: wait for m_axi_bvalid.
IAR	I-bus AXI read address phase: m_axi_araddr set to I-fetch PC; wait for arready.
IR	I-bus AXI read data phase: wait for m_axi_rvalid; return instruction to core I-path.

Chapter 8. Electrical and Timing Characteristics

8.1. Clock and Reset

8.1.1. Clock Domains

The SoC has two independent clock domains.

Table 25. Clock Domain Summary

Domain	Source	Nominal Frequency	Clocks
System (`clk`)	External pin	100 MHz (parameterisable via `CLK_FREQ`)	`jv32_core`, TCM SRAM, AXI crossbar, UART, CLIC, Magic, debug system clock path
TAP (`tck` / `tap_tck`)	External pin (`USE_CJTAG=0`) or internally generated BUFG (`USE_CJTAG=1`)	≤ 10 MHz (`USE_CJTAG=0`), ≤ $f_{sys}/6$ (`USE_CJTAG=1`)	`jtag_tap`, `jv32_dtm` TAP state machine

The two domains are **fully asynchronous**. All signals crossing the boundary pass through two-stage synchroniser chains in `jv32_dtm` (see `ASYNC_REG`-attributed registers in the RTL).

8.1.2. System Clock (`clk`)

The system clock drives the processor pipeline, TCM, peripherals, and the system-clock half of the debug transport module.

Table 26. System Clock Specifications

Parameter	Value	Description
Pin	`clk`	Single-ended clock input
Nominal frequency	100 MHz (default)	Parameterisable via `CLK_FREQ`; affects UART baud divisor computation
Minimum frequency	No lower limit specified	Must satisfy `CLK_FREQ / BAUD_RATE ≥ 4` for correct UART operation

Parameter	Value	Description
Maximum frequency	Technology-dependent	FPGA (KU5P): 50 MHz demonstrated; ASIC (FreePDK45): 80 MHz post-PnR sign-off

8.1.3. TAP Clock — USE_CJTAG=0 (4-wire JTAG)

In 4-wire JTAG mode the TAP clock is the external `jtag_pin0_tck_i` pin, driven directly by the debug probe.

Table 27. 4-wire JTAG TAP Clock Specifications

Parameter	Value	Description
Source	`jtag_pin0_tck_i` (external pin)	TCK driven by the JTAG probe; asynchronous to `clk`
Maximum frequency	f_{sys} / 2 (hard limit), f_{sys} / 4 recommended	Must be slow enough for the two-stage synchronisers in `jv32_dtm` to resolve metastability; ≤ 10 MHz typical for the FPGA build
Relationship to `clk`	Fully asynchronous	`set_clock_groups -asynchronous` between `jtag_tck` and the system clock domain
Constraint file (FPGA)	`fpga/scripts/constraints.xdc` (primary clock), `fpga/scripts/constraints_jtag.xdc` (I/O delays)	TDI: `set_input_delay -clock jtag_tck`; TDO: `set_output_delay -clock jtag_tck -clock_fall` (negedge launch)

8.1.4. TAP Clock — USE_CJTAG=1 (2-wire cJTAG)

In 2-wire cJTAG mode the external `jtag_pin0_tck_i` pin carries the raw TCKC signal. The `cjtag_bridge` module samples TCKC using the system clock and regenerates an internal TAP clock (`tck_int`) as a registered pulse, which is then buffered through a `BUFG` (`u_bufg_tck`) to become `tck_o`.

Table 28. 2-wire cJTAG TAP Clock Specifications (USE_CJTAG=1)

Parameter	Value	Description
TCKC source	`jtag_pin0_tck_i` (external pin)	2-wire cJTAG clock from probe; asynchronous to `clk`

Parameter	Value	Description
TAP clock source (`tck_o`)	`cjtag_bridge` BUFG output (`u_bufg_tck/O`)	`tck_int` register (clocked by `clk`) → BUFG; the TAP clock is **derived from the system clock**
Maximum TCKC frequency	f_{sys} / 6	`cjtag_bridge` requires ≥ 3 system clock cycles per TCKC half-period for two-stage sync + edge detection; at 100 MHz system clock, TCKC ≤ 16.7 MHz; at 50 MHz (FPGA), TCKC ≤ 8.3 MHz
Relationship to `clk`	Downstream of `clk` (generated by `clk`-clocked flip-flop)	`set_clock_groups -physically_exclusive` suppresses TIMING-3 (AR#63774); `set_clock_groups -asynchronous` separates the domains for STA
Constraint file (FPGA)	`fpga/scripts/constraints_cjtag.xdc` (implementation-only)	`create_clock` targets the `BUFG/O` pin of `u_bufg_tck`, not the external TCKC pin
Input synchronisers in bridge	2-stage (`ASYNC_REG = "TRUE"`)	TCKC and TMSC are both sampled through 2-stage synchronisers before any combinational or registered logic uses them

In simulation the `BUFG` macro is replaced by a plain `assign tck_o = tck_int` (`` \`ifdef SYNTHESIS ``). The TAP clock is therefore the combinational value of `tck_int` in simulation and the BUFG-buffered version in synthesis.

8.1.5. Reset Architecture

The SoC implements an **asynchronous-assert / synchronous-deassert** reset pattern. Two independent sources can assert reset: the external `rst_n` pin and the debug non-debug-module reset (`dbg_ndmreset`) asserted by the RISC-V Debug Module via JTAG.

8.1.5.1. Reset signal flow

```
// jv32_soc.sv
assign rst_n_pre = rst_n & ~dbg_ndmreset;   // (1) OR of both reset sources

always_ff @(posedge clk or negedge rst_n_pre) begin
    if (!rst_n_pre) begin
        rst_sync_ff1 <= 1'b0;               // (2) async assert: both FFs cleared immediately
        rst_sync_ff2 <= 1'b0;
    end else begin
```

```
        rst_sync_ff1 <= 1'b1;              // (3) synchronous deassert: propagates over 2 clk cycles
        rst_sync_ff2 <= rst_sync_ff1;
    end
end

assign soc_rst_n = rst_sync_ff2;           // (4) clean synchronised reset to all SoC logic
```

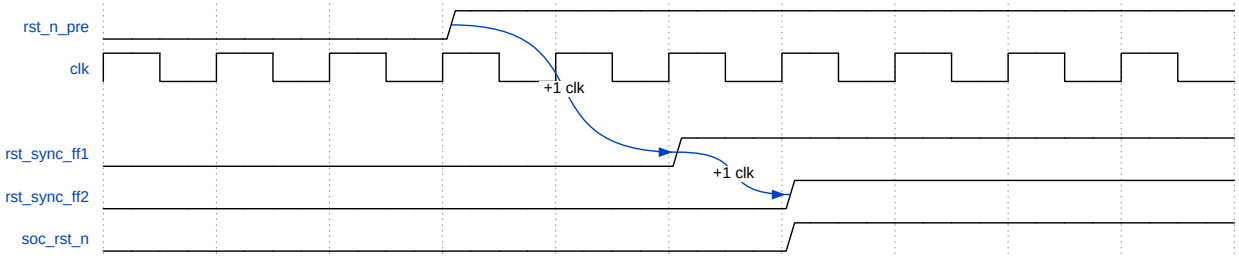


Figure 1. Reset de-assertion timing (2-cycle synchroniser)

8.1.5.2. Reset source behaviour

Table 29. Reset Sources

Source	Assertion	Description
`rst_n`	Asynchronous (active-low)	External system reset pin; asserted immediately by hardware; de-asserted synchronously through the 2-FF chain
`dbg_ndmreset`	Asynchronous (active-high) from `jv32_dtm`	Non-debug-module reset issued over JTAG (`dmcontrol.ndmreset`); resets the SoC while the debug module itself remains active; `rst_n_pre = rst_n & ~dbg_ndmreset`
`dbg_hartreset`	Synchronous, core only	Hart reset signal from `jv32_dtm`; resets the `jv32_core` pipeline registers only, without affecting peripherals or the debug infrastructure

8.1.5.3. Reset domain coverage

`soc_rst_n` (the synchronised output of the 2-FF chain) is distributed to:

- `jv32_top` — processor pipeline, TCM SRAM control registers, AXI master/slave
- `axi_clic` — CLIC/CLINT registers (`mtime`, `mtimecmp`, `msip`, interrupt pending)
- `axi_uart` — UART TX/RX FIFOs and baud rate generator
- `axi_xbar` — AXI crossbar state

The JTAG TAP (`jtag_tap`) and `jv32_dtm` are reset by `rst_n` **directly** (not through the synchroniser), so debug infrastructure is available as soon as the external reset is released, independent of the 2-cycle delay on `soc_rst_n`. `jtag_ntrst_i` provides an additional TAP-only reset.

8.1.5.4. Reset timing requirements

Table 30. Reset Timing Requirements

Parameter	Minimum	Description
`rst_n` assert width	1 ns	No minimum pulse width; async clear is edge-sensitive on `negedge rst_n_pre`
`rst_n` deassert hold (before first `clk` rising edge used for computation)	2 clock cycles	Two `clk` rising edges are required before `soc_rst_n` deasserts and SoC logic begins operating
`rst_n` setup before `clk` rising edge	Not required	Deassertion is sampled synchronously; no setup requirement on the deassert edge
`dbg_ndmreset` minimum width	1 `clk` cycle	Must remain high for at least one complete `clk` cycle to ensure the async clear of `rst_sync_ff1/ff2` is captured

The `rst_n_pre` net is a 2-input LUT driving the asynchronous clear (`CLR`) of `rst_sync_ff1` and `rst_sync_ff2`. In FPGA implementations this is an intentional asynchronous-reset pattern; the path is marked as a false path to suppress the Vivado LUTAR-1 methodology warning (`fpga/scripts/constraints.xdc`).

8.2. Timing Parameters

Table 31. Timing Characteristics (100 MHz, Tclk = 10 ns)

Parameter	Min	Max	Description
Setup time (AXI inputs)	2.0 ns	—	Input setup before clock edge
Hold time (AXI inputs)	1.0 ns	—	Input hold after clock edge
Clock-to-output (AXI outputs)	—	8.0 ns	Output valid after clock edge
TCM read latency	1 cycle	—	Registered SRAM output
Load-to-use latency	2 cycles	—	1 EX cycle + 1 stall cycle
Branch flush penalty	0 / 1 cycle	—	0 cycles on correct BTFNT prediction; 1 cycle on misprediction or when `BP_EN=0` and branch is taken

Parameter	Min	Max	Description
Interrupt entry latency	3 cycles	10 cycles	Depends on pipeline state and outstanding transactions

8.3. Top-Level Parameters

Table 32. `jv32_soc` Configurable Parameters

Parameter	Default	Description
`CLK_FREQ`	`100_000_000`	System clock frequency in Hz; used to compute UART baud divisor
`BAUD_RATE`	`115_200`	UART baud rate (must satisfy `CLK_FREQ / BAUD_RATE >= 4`)
`UART_FIFO_DEPTH`	`16`	TX and RX FIFO depth (same value for both; must be a power-of-2)
`RV32E_EN`	`0`	`1`=RV32E reduced register file (16 GPRs); `0`=RV32I full register file (32 GPRs). Also gates `misa.E`.
`RV32M_EN`	`1`	`1`=M-extension enabled (MUL/DIV/REM instructions legal); `0`=all multiply/divide instructions trap as illegal
`JTAG_EN`	`1`	`1`=JTAG debug port and RISC-V Debug Module instantiated; `0`=all debug outputs tied off, JTAG pins ignored
`USE_CJTAG`	`0`	`0`=4-wire JTAG (TCK/TMS/TDI/TDO); `1`=2-wire cJTAG IEEE 1149.7 OScan1 (TCKC/TMSC); see Clock and Reset for clock architecture
`JTAG_IDCODE`	`32'h1DEAD3FF`	JTAG TAP `IDCODE` register value returned by the DTM on IR=`IDCODE` (0x01) scan
`N_TRIGGERS`	`2`	Number of hardware breakpoint/watchpoint trigger registers (`tdata1`/`tdata2` pairs); range 0..4
`AMO_EN`	`1`	`1`=full A-extension (LR/SC + all AMO{SWAP,ADD,XOR,AND,OR,MIN,MAX,MINU,MAXU}.W); `0`=LR.W/SC.W only
`FAST_MUL`	`1`	`1`=single-cycle combinatorial multiply; `0`=multi-cycle serial multiply (CLZ-reduced; 1–32 cycles)
`MUL_MC`	`1`	When `FAST_MUL=0`: `1`=CLZ-based iteration count reduction; `0`=always 32 iterations. Has no effect when `FAST_MUL=1`.

Parameter	Default	Description
FAST_DIV	0	1=single-cycle combinatorial divide; 0=CLZ-reduced serial divide (1–32 cycles)
FAST_SHIFT	1	1=single-cycle barrel shift; 0=iterative shift (up to 31 cycles)
BP_EN	1	1=BTFNT static branch predictor enabled (backward branches predicted taken, forward branches predicted not-taken; 0-cycle penalty on correct prediction, 1-cycle on misprediction); 0=all branches predicted not-taken (1-cycle penalty on every taken branch)
RAS_EN	1	1=2-entry Return Address Stack enabled (zero-penalty function returns via RAS predict); 0=JALR always causes a 1-cycle EX-stage redirect. Automatically disabled when RV32EC=1.
IBUF_EN	1	1=2-entry instruction prefetch buffer enabled (hides 1-cycle TCM latency on sequential fetch); 0=disabled. Automatically disabled when RV32EC=1 (RV32E_EN=1).
ZCMP_EN	1	1=Zcmp extension enabled (cm.push, cm.pop, cm.popret, cm.mva01s, cm.mvsa01); 0=these instructions trap as illegal.
RV32B_EN	1	1=Zba/Zbb/Zbs B-extension enabled (SH1ADD/SH2ADD/SH3ADD, bit-manipulation, etc.); 0=all B-extension instructions trap as illegal (synthesised away).
IRAM_SIZE	131072	IRAM size in bytes (power-of-2; also sets TCM address decode mask)
DRAM_SIZE	131072	DRAM size in bytes (power-of-2)
BOOT_ADDR	32'h8000_0000	Reset PC (word-aligned). Must equal IRAM_BASE for the core to boot from IRAM.
IRAM_BASE	32'h8000_0000	IRAM TCM base address and address-decode base. Normally equals BOOT_ADDR.
DRAM_BASE	32'h9000_0000	DRAM base address and TCM address decode start

Chapter 9. Programming Guide

9.1. Boot Sequence

The processor boots from `BOOT_ADDR` (default `0x8000_0000` = IRAM base). The reset vector is hardwired to `BOOT_ADDR & ~3` (word-aligned).

For simulation, the typical boot flow is:

1. ELF loaded into TCM via DPI (`mem_write_byte`) before simulation starts
2. Simulation starts, core fetches from `BOOT_ADDR` (mapped to IRAM, 1-cycle latency)
3. Startup code (`startup.S`) initialises stack pointer to top of DRAM
4. Hardware CSRs initialised: `mstatus`, `mtvec`, `mie`
5. Peripherals initialised (UART baud rate is compile-time; CLIC `mtimecmp` programmed)
6. `mstatus.MIE` set to enable interrupts
7. Jump to `main()`

9.2. Stack Configuration

Default linker script maps DRAM to data/BSS/stack:

```
Stack top: DRAM_BASE + DRAM_SIZE - 4    (e.g. 0x9001_FFFC)
Stack grows downward into DRAM
Data/BSS:  DRAM_BASE (loaded by startup.S from IRAM)
```

9.3. Interrupt Handling

9.3.1. Setting Up Interrupts

```c
#include "csr.h"

extern void trap_entry(void);  // defined in startup.S

void setup_interrupts(void) {
    // Set trap vector (direct mode: mtvec points to handler)
    write_csr(mtvec, (uint32_t)trap_entry);

    // Enable M-mode interrupts in mstatus
    set_csr(mstatus, 0x8);  // MIE bit

    // Enable timer + external + software interrupts in mie
    set_csr(mie, (1<<7) | (1<<11) | (1<<3));  // MTIE | MEIE | MSIE
```

```
    // Program CLIC mtimecmp for periodic 1ms timer (100 MHz)
    volatile uint32_t *mtime_lo    = (uint32_t *)0x02004000;
    volatile uint32_t *mtime_hi    = (uint32_t *)0x02004004;
    volatile uint32_t *mtimecmp_lo = (uint32_t *)0x02004008;
    volatile uint32_t *mtimecmp_hi = (uint32_t *)0x0200400C;

    // Set mtimecmp_hi first to avoid spurious IRQ
    *mtimecmp_hi = 0xFFFFFFFF;
    uint64_t now = ((uint64_t)*mtime_hi << 32) | *mtime_lo;
    uint64_t cmp = now + 100000;  // 1 ms @ 100 MHz
    *mtimecmp_lo = (uint32_t)cmp;
    *mtimecmp_hi = (uint32_t)(cmp >> 32);
}
```

9.3.2. Trap Handler

```
void trap_handler(uint32_t mcause_val) {
    if (mcause_val & 0x80000000) {
        // Interrupt
        uint32_t cause = mcause_val & 0x7FFFFFFF;
        switch (cause) {
        case 7:   // Machine timer interrupt
            handle_timer_irq();
            break;
        case 11:  // Machine external interrupt (CLIC)
            handle_clic_irq();
            break;
        case 3:   // Machine software interrupt
            handle_sw_irq();
            break;
        }
    } else {
        // Synchronous exception
        handle_exception(mcause_val);
    }
}

void handle_timer_irq(void) {
    // Clear by advancing mtimecmp
    volatile uint32_t *cmp_lo = (uint32_t *)0x02004008;
    volatile uint32_t *cmp_hi = (uint32_t *)0x0200400C;
    uint64_t cmp = ((uint64_t)*cmp_hi << 32) | *cmp_lo;
    cmp += 100000;  // next 1 ms
    *cmp_hi = 0xFFFFFFFF;  // prevent spurious IRQ during update
    *cmp_lo = (uint32_t)cmp;
    *cmp_hi = (uint32_t)(cmp >> 32);
}

void handle_clic_irq(void) {
    // CLIC vectored entry: mcause = 0x8000_000B, mintstatus.mil = this IRQ's level.
    // Scan CLICINT[n].ip to identify the source; IP auto-clears when ext_irq_i[n] deasserts.
    for (int n = 0; n < 16; n++) {
        volatile uint32_t *clicint = (uint32_t *)(0x02001000 + n * 4);
        if (*clicint & 0x1) {
```

```
            // Source n is pending — handle it here.
        }
    }
    // Software-assisted tail-chain: check for next pending IRQ via mnxti.
    // If a higher-priority IRQ arrived during this handler, chain directly to it.
    uintptr_t next = read_csr(mnxti);
    if (next) {
        // Claim it (sets mcause, mintstatus.mil, re-enables MIE) and branch.
        write_csr(mnxti, 0);
        ((void (*)(void))next)();
    }
    // Hardware tail-chain: if another CLIC IRQ above mintthresh is pending
    // when this handler's mret executes, the CPU redirects to mtvt+4*id
    // automatically — no code change needed.
}
```

9.4. UART Usage

```
#define UART_DATA   (*(volatile uint32_t *)0x20010000)
#define UART_STATUS (*(volatile uint32_t *)0x20010004)
#define UART_IE     (*(volatile uint32_t *)0x20010008)
#define UART_IS     (*(volatile uint32_t *)0x2001000C)

// Poll-mode transmit
void uart_putc(char c) {
    while (UART_STATUS & 0x1);  // wait until TX not full
    UART_DATA = c;
}

// Poll-mode receive
char uart_getc(void) {
    while (!(UART_STATUS & 0x4));  // wait until RX not empty
    return (char)(UART_DATA & 0xFF);
}

// Print string
void uart_puts(const char *s) {
    while (*s) uart_putc(*s++);
}
```

9.5. Simulation Debugging

9.5.1. Magic Device Console

```
#define MAGIC_CON (*(volatile uint32_t *)0x40000000)
#define MAGIC_EXIT (*(volatile uint32_t *)0x40000004)

void debug_putc(char c) { MAGIC_CON = c; }
void debug_puts(const char *s) { while (*s) debug_putc(*s++); }
void sim_pass(void) { MAGIC_EXIT = 1; }              // HTIF: exit(0)
void sim_fail(int code) { MAGIC_EXIT = (code << 1) | 1; }  // HTIF: exit(code)
```

9.5.2. Waveform Capture

Generate VCD waveform for GTKWave analysis:

```
make WAVE=1 rtl-hello
gtkwave build/tb_jv32_soc.vcd
```

Key signals to observe:

- **Pipeline**: `jv32_core.pc_if`, `jv32_core.pc_ex`, `jv32_core.pc_wb`
- **TCM**: `jv32_top.imem_req_valid`, `jv32_top.imem_resp_valid`, `jv32_top.dmem_req_valid`
- **AXI master**: `jv32_top.m_axi_arvalid`, `jv32_top.m_axi_rvalid`
- **Interrupts**: `jv32_soc.timer_irq`, `jv32_soc.clic_irq`
- **Heartbeat**: `jv32_soc.heartbeat_o` — toggles every 2^24 retired instructions; a slow square wave indicating the core is making forward progress

9.5.3. Instruction Trace

Software and RTL traces can be compared for regression:

```
make compare-hello
diff sim_trace.txt rtl_trace.txt
```

Trace format: `<cycle> <pc> <hex_instruction> <disassembly>`

9.6. Performance Optimisation Tips

1. **Minimise out-of-TCM accesses**: Place hot code in IRAM (`BOOT_ADDR` region) and data in DRAM (`DRAM_BASE` region) — both 1-cycle access
2. **Eliminate load-use stalls**: Reorder instructions so a load result is not used in the immediately following instruction
3. **Use fast ALU options**: `FAST_MUL=1`, `FAST_DIV=1`, `FAST_SHIFT=1` for single-cycle operations (area trade-off)
4. **Keep stack in DRAM**: All stack frames in DRAM ensure 1-cycle load/store latency
5. **Avoid IRAM D-access contention**: Keep data-heavy code paths away from IRAM address range to avoid I-fetch stalls from priority resolution

Chapter 10. Conclusion

This datasheet provides a complete reference for the JV32 RV32IMAC SoC, covering its 3-stage TCM-based pipeline architecture, memory map, peripheral specifications, AXI interconnect, and programming guide. For additional information refer to:

- **RISC-V Specifications**: https://riscv.org/technical/specifications/
- **AMBA AXI4-Lite Specification**: https://developer.arm.com/documentation/ihi0022/latest/
- **Project Repository Source**: `/home/kuoping/Projects/jv32/`
- **RTL Entry Point**: `rtl/jv32_soc.sv`, `rtl/jv32/jv32_top.sv`

Appendix A: Register Quick Reference

A.1. CLIC Registers

Address	Name	Access	Reset Value
0x0200_0000	MSIP	R/W	0x0000_0000
0x0200_4000	MTIME_LO	R/W	0x0000_0000
0x0200_4004	MTIME_HI	R/W	0x0000_0000
0x0200_4008	MTIMECMP_LO	R/W	0xFFFF_FFFF
0x0200_400C	MTIMECMP_HI	R/W	0xFFFF_FFFF
0x0200_1000 + n×4	CLICINT[n] (n=0..15)	R/W	0x0000_0000

A.2. UART Registers

Address	Name	Access	Reset Value
0x2001_0000	DATA	R/W	0x0000_0000
0x2001_0004	STATUS	R/O	0x0000_0000 (TX FIFO empty, TX ready)
0x2001_0008	IE	R/W	0x0000_0000
0x2001_000C	IS	R/O	0x0000_0002 (TX empty)
0x2001_0010	LEVEL	R/W	0x0000_0000
0x2001_0014	CTRL	R/W	0x0000_0000
0x2001_0018	CAPABILITY	R/O	0x0001_1010 (version=1, TX depth=16, RX depth=16 with default FIFO_DEPTH=16)

A.3. Magic Device Registers

Address	Name	Access	Reset Value
0x4000_0000	CONSOLE	W/O	—

Address	Name	Access	Reset Value
0x4000_0004	EXIT	W/O	—

Appendix B: Interrupt Vector Table

Table 33. Machine-Mode Interrupt Vectors (mcause encoding)

Code	Type	Description	Source
0	Exception	Instruction address misaligned	Core
1	Exception	Instruction access fault	I-fetch AXI SLVERR/DECERR
2	Exception	Illegal instruction	Core decode
3	Exception	Breakpoint	EBREAK
4	Exception	Load address misaligned	Core
5	Exception	Load access fault	D-bus AXI SLVERR/DECERR
6	Exception	Store/AMO address misaligned	Core
7	Exception	Store/AMO access fault	D-bus AXI SLVERR/DECERR
8	Exception	ECALL from U-mode	(not reached in M-only system)
11	Exception	ECALL from M-mode	ECALL instruction
`0x8000_0003`	Interrupt	Machine software interrupt	CLIC MSIP[0]
`0x8000_0007`	Interrupt	Machine timer interrupt	CLIC `mtime >= mtimecmp`
`0x8000_000B`	Interrupt	Machine external interrupt	CLIC any `CLICINT[n].ie & ip` with level > mintthresh

`mcause[31]=1` indicates interrupt; `mcause[30:0]` is the cause code.

Appendix C: Pin Descriptions

C.1. Clock and Reset

Table 34. Clock and Reset Pins

Signal	I/O	Width	Description
`clk`	Input	1	System clock (100 MHz nominal)
`rst_n`	Input	1	Asynchronous active-low reset

C.2. UART Interface

Table 35. UART Pins

Signal	I/O	Width	Description
`uart_tx_o`	Output	1	UART transmit data
`uart_rx_i`	Input	1	UART receive data (asynchronous)

C.3. External Interrupt Interface

Table 36. External Interrupt Pins

Signal	I/O	Width	Description
`ext_irq_i[15:0]`	Input	16	External interrupt request lines, routed to CLIC `CLICINT[n].ip`; active-high, level-sensitive

C.4. TCM AXI Slave Interface

Table 37. TCM Slave Port Pins (exposed as `s_tcm_*` on `jv32_soc`)

Signal	I/O	Width	Description
`s_tcm_araddr[31:0]`	Input	32	Read address
`s_tcm_arvalid`	Input	1	Read address valid
`s_tcm_arready`	Output	1	Read address ready
`s_tcm_rdata[31:0]`	Output	32	Read data

Signal	I/O	Width	Description
s_tcm_rresp[1:0]	Output	2	Read response
s_tcm_rvalid	Output	1	Read data valid
s_tcm_rready	Input	1	Read data ready
s_tcm_awaddr[31:0]	Input	32	Write address
s_tcm_awvalid	Input	1	Write address valid
s_tcm_awready	Output	1	Write address ready
s_tcm_wdata[31:0]	Input	32	Write data
s_tcm_wstrb[3:0]	Input	4	Write byte strobes
s_tcm_wvalid	Input	1	Write data valid
s_tcm_wready	Output	1	Write data ready
s_tcm_bresp[1:0]	Output	2	Write response
s_tcm_bvalid	Output	1	Write response valid
s_tcm_bready	Input	1	Write response ready

C.5. JTAG / cJTAG Debug Interface

Active only when JTAG_EN=1. Pin roles differ between USE_CJTAG=0 (4-wire JTAG) and USE_CJTAG=1 (2-wire cJTAG).

Table 38. JTAG / cJTAG Debug Pins

Signal	I/O	Width	Description
jtag_ntrst_i	Input	1	TAP reset (active-low); resets the JTAG TAP state machine independently of rst_n. Tie high if not used.

Signal	I/O	Width	Description
jtag_pin0_tck_i	Input	1	USE_CJTAG=0: TAP clock (TCK), driven by debug probe, asynchronous to clk. USE_CJTAG=1: cJTAG serial clock (TCKC); asynchronous to clk; internally sampled by 2-stage synchroniser in cjtag_bridge. Maximum frequency = $f_{sys}/6$.
jtag_pin1_tms_i	Input	1	USE_CJTAG=0: TAP Mode Select (TMS). USE_CJTAG=1: cJTAG data (TMSC) input direction; driven by probe during scan-in.
jtag_pin1_tms_o	Output	1	USE_CJTAG=0: not driven (tie jtag_pin1_tms_oe low). USE_CJTAG=1: TMSC output direction; valid when jtag_pin1_tms_oe=1 (SoC driving TMSC during scan-out).
jtag_pin1_tms_oe	Output	1	TMSC output-enable (active-high). Drive the OE of an external IOBUF on TMSC. In USE_CJTAG=0 mode this is always 0.
jtag_pin2_tdi_i	Input	1	USE_CJTAG=0: TAP Data In (TDI). USE_CJTAG=1: not used (internal cJTAG shift path is fully serialised over TMSC).
jtag_pin3_tdo_o	Output	1	USE_CJTAG=0: TAP Data Out (TDO); valid on falling edge of TCK. USE_CJTAG=1: not used; cJTAG scan-out uses TMSC.
jtag_pin3_tdo_oe	Output	1	TDO output-enable (active-high). Drive the OE of an external tristate buffer on TDO. In USE_CJTAG=0 mode this follows the TAP shift state.

C.6. Trace Interface

Present in simulation only (excluded from synthesis via `ifndef SYNTHESIS`). All outputs change on the rising edge of `clk` and are qualified by trace_valid. Connect to the testbench trace logger.

Table 39. Trace Output Pins

Signal	I/O	Width	Description
trace_en	Input	1	Run-time trace enable gate; when low, trace_valid stays deasserted
trace_valid	Output	1	Strobe: high for one clk cycle when a new instruction has retired
trace_reg_we	Output	1	Register file write enable for the retired instruction
trace_pc	Output	32	PC of the retired instruction
trace_rd	Output	5	Destination register index of the retired instruction
trace_rd_data	Output	32	Value written to rd (valid when trace_reg_we=1)
trace_instr	Output	32	32-bit expanded instruction word (after RVC expansion)
trace_mem_we	Output	1	Memory write-enable for the retired instruction (store)
trace_mem_re	Output	1	Memory read-enable for the retired instruction (load)
trace_mem_addr	Output	32	Memory address of the load/store
trace_mem_data	Output	32	Data written to memory (store) or loaded from memory (load)
trace_irq_taken	Output	1	Strobe: high for one cycle when an interrupt is taken (pipeline redirected to handler)
trace_irq_cause	Output	32	mcause value captured at interrupt entry
trace_irq_epc	Output	32	mepc value captured at interrupt entry (pre-empted PC)
trace_irq_store_we	Output	1	Interrupt pipeline — memory write-enable for interrupt-triggered store (e.g. stack save)
trace_irq_store_addr	Output	32	Interrupt pipeline — store address
trace_irq_store_data	Output	32	Interrupt pipeline — store data

C.7. Branch Predictor Observability (perf_bp_*)

One-cycle pulse outputs that count branch-predictor events. Active only with BP_EN=1; tied to 0 otherwise. Intended for performance counters in a testbench or external logic analyser.

Table 40. Branch Predictor Observability Pins

Signal	I/O	Width	Description
perf_bp_branch	Output	1	A conditional branch retired this cycle
perf_bp_taken	Output	1	The conditional branch was taken
perf_bp_mispred	Output	1	Branch prediction was wrong (EX-stage redirect)
perf_bp_jal	Output	1	A JAL instruction retired this cycle
perf_bp_jal_miss	Output	1	JAL was not pre-decoded (caused an EX-stage redirect)
perf_bp_jalr	Output	1	A JALR instruction retired this cycle (always causes an EX-stage redirect)

C.8. Heartbeat Output

heartbeat_o is a glitch-free single-bit output derived directly from bit 24 of the minstret counter inside jv32_csr. It requires no external logic, no clock-domain crossing, and is always present regardless of BP_EN.

Table 41. Heartbeat Pin

Signal	I/O	Width	Description
heartbeat_o	Output	1	minstret[24] — toggles every 2^24 retired instructions. One complete low→high→low period spans 2 × 2^24 × CPI clock cycles (≈ 42 M cycles at CPI 2.5, 80 MHz → ~530 ms). Can be connected to an LED or logic analyser trigger to confirm the core is retiring instructions.

The toggle period in seconds is:

$$T = \frac{2 \times 2^{24} \times \overline{\mathrm{CPI}}}{f_{\mathrm{clk}}}$$

At the default 80 MHz FPGA clock with CoreMark CPI ≈ 1.25, T ≈ 420 ms.

C.9. External AXI Master Interface (ext_axi_*)

The external AXI master port carries transactions that fall outside all TCM and peripheral apertures (crossbar slave 3 catch-all). Connect to an external memory or MMIO fabric, or tie ext_axi_arready, ext_axi_rvalid, ext_axi_awready, ext_axi_wready, ext_axi_bvalid all to 0 if no external master is needed (the core will stall until reset).

Table 42. External AXI Master Pins

Signal	I/O	Width	Description
ext_axi_araddr[31:0]	Output	32	Read address
ext_axi_arvalid	Output	1	Read address valid
ext_axi_arready	Input	1	Read address ready
ext_axi_rdata[31:0]	Input	32	Read data
ext_axi_rresp[1:0]	Input	2	Read response (OKAY/SLVERR/DECERR)
ext_axi_rvalid	Input	1	Read data valid
ext_axi_rready	Output	1	Read data ready
ext_axi_awaddr[31:0]	Output	32	Write address
ext_axi_awvalid	Output	1	Write address valid
ext_axi_awready	Input	1	Write address ready
ext_axi_wdata[31:0]	Output	32	Write data
ext_axi_wstrb[3:0]	Output	4	Write byte strobes
ext_axi_wvalid	Output	1	Write data valid
ext_axi_wready	Input	1	Write data ready
ext_axi_bresp[1:0]	Input	2	Write response
ext_axi_bvalid	Input	1	Write response valid

Signal	I/O	Width	Description
ext_axi_bready	Output	1	Write response ready